

Migración a Odoo 20

RIDERY

Plan detallado · desde Odoo 16

Mornix · Agosto 2026

Detalle técnico derivado de la migración piloto de `l10n_ve_nimetrix`

Propósito de este documento

Toma el cronograma acordado para RIDERY y le agrega el detalle de ejecución: qué se hace cada mes, con qué se verifica, y qué puede salir mal. Las roturas del Anexo A y las herramientas del Anexo B no son teóricas — salen de una migración real ejecutada de punta a punta sobre código de esta misma casa.

Alcance de lo verificado

La migración piloto fue de **v18 a v20**. RIDERY está en **v16**, o sea **cuatro saltos** (17, 18, 19, 20). A lo verificado hay que sumarle las roturas de v16→v18, marcadas en el Anexo A como **esperadas**: están documentadas por Odoo pero todavía no confirmadas contra código de RIDERY.

Situación de Odoo 20 al día de hoy

Hecho	Estado a agosto 2026
Rama 20.0	No existe todavía. Se trabaja sobre master
Versión que declara master	19.5 alpha
Ramas saas publicadas	hasta saas-19.4
Release esperado de 20.0	~octubre 2026

El patrón histórico de Odoo es saas-X.1 ... saas-X.4 y después (X+1).0 . Con saas-19.4 ya publicada y master en 19.5, la ramificación de 20.0 en octubre es consistente. **El cronograma de RIDERY es compatible con esa fecha.**

Riesgos del cronograma

Cinco puntos del plan acordado que conviene reforzar. Ninguno lo invalida; todos son manejables si se atienden temprano.

Riesgo 1 — Los módulos de terceros no aparecen en el plan

Es el techo real de la fecha de salida

En el proyecto piloto,
el 30 % del código eran módulos de vendors externos
: 50 módulos, 85.205 líneas. Ninguno tiene versión v20 antes de que Odoo libere v20, y su fecha no la controlamos nosotros.

Si RIDERY usa módulos de pago — temas, integraciones, conectores — hay que **contactar a cada proveedor en agosto**
, junto con el assessment, no en diciembre. Un vendor que responde «para marzo» mueve el pase a producción aunque todo lo demás esté listo.

Acción: incorporar al assessment de agosto un listado de módulos de terceros con su proveedor, y abrir el contacto de inmediato. Por cada uno hay tres salidas: esperar su v20, portarlo nosotros (revisando licencia), o reemplazar la funcionalidad.

Riesgo 2 — Contable y nómina en la misma ventana

El plan pone «Generación Localización Contable V20» y «Generación Localización Nómina V20» juntas en octubre-noviembre. Como referencia de tamaño, la localización de nómina que esta casa mantiene para otro cliente son **35 módulos, 17.625 líneas en el núcleo y 254 pruebas**. Eso no se rehace en dos meses compartidos con la contable.

Acción: decidir en agosto si nómina se migra o se pospone a una segunda fase. Si va en la misma ventana, necesita equipo propio.

Riesgo 3 — Sin colchón entre la beta y la construcción

El plan revisa la beta en septiembre y empieza a construir en octubre. Si la beta revela cambios estructurales, ese mes se consume en rehacer.

Y los hay: el piloto encontró que en v20 **el ORM se reorganizó en paquetes, el modelo de seguridad se unificó y la API de consultas SQL se rehizo entera**. No son ajustes cosméticos.

Acción: adelantar la revisión técnica a agosto sobre `master`, que ya es sustancialmente lo que será 20.0. No hace falta esperar a la beta para encontrar el 80 % de las roturas — el piloto encontró más de veinte trabajando sobre `master`.

Riesgo 4 — El assessment no incluye el acoplamiento real

Los manifests mienten

En el piloto se encontraron
42 campos usados sin declarar la dependencia, en 9 módulos
. Un módulo generaba un reporte contra un campo de otro módulo que no declaraba: instalaba limpio y fallaba recién al generar el reporte.

El acoplamiento real decide el **orden de migración**
. En el piloto cambió el plan: lo que parecía un área lateral resultó ser el cimiento del resto.

Acción: incluir en el assessment de agosto el análisis de dependencias no declaradas (Anexo B). Es automático y toma minutos.

Riesgo 5 — El pase a producción no tiene ensayo ni vuelta atrás

El plan contempla migración de datos reales y pruebas finales, pero no un **ensayo del pase** ni un procedimiento de reversión escrito.

Acción: agregar en enero un ensayo completo del corte sobre copia de producción, cronometrado, y documentar cómo se revierte. Restaurar un respaldo revierte código y esquema, pero borra todo lo que el cliente haya cargado desde que se tomó: en horario laboral eso es pérdida de datos real.

Plan mes a mes

Las filas marcadas con + son adiciones al plan acordado.

Agosto 2026

Assessment

Actividad	Entregable verificable	Resp.
Assessment de funcionalidades implementadas	Catálogo funcional	RIDERY-MORNIX
Definir quién documenta cada funcionalidad	Matriz de responsabilidades	RIDERY-MORNIX
+ Inventario técnico de módulos	Listado con líneas de Python, XML y JavaScript por módulo	MORNIX
+ Clasificación por origen	Propio / tercero de pago / OCA / sin clasificar	MORNIX
+ Contacto con venders de terceros	Respuesta escrita de cada uno sobre su v20	MORNIX-RIDERY
+ Análisis de acoplamiento no declarado	Grafo real de dependencias y orden de migración	MORNIX
+ Confirmar qué rama refleja producción	Commit desplegado, verificado contra el servidor	RIDERY
+ Solicitar volcado anonimizado	Copia restaurable	RIDERY
+ Revisión técnica anticipada sobre master	Catálogo preliminar de roturas	MORNIX

Por qué el volcado se pide en agosto

Sobre base limpia casi todo pasa. Los problemas caros — datos históricos, secuencias fiscales, saldos — solo aparecen sobre datos reales. Pedirlo temprano evita descubrir en enero que no lo dan, o que hay que anonimizarlo y nadie presupuestó ese trabajo.

Septiembre 2026

Evaluación de v20

Actividad	Entregable verificable	Resp.
Revisar beta de Odoo 20	Informe de evaluación	MORNIX
Listado de funcionalidades nuevas	Documento comparativo	MORNIX
+ Catálogo de roturas cerrado	Bitácora con síntoma, versión y solución por cada una	MORNIX
+ Migración de un módulo piloto	Un módulo real de RIDERY instalando y con pruebas en verde	MORNIX
+ Estimación recalibrada	Estimación por módulo, medida sobre el piloto	MORNIX
+ Decisión sobre nómina	Entra en esta fase o se pospone	RIDERY-MORNIX

El piloto es lo que convierte el plan en estimación

Migrar un módulo real de RIDERY antes de comprometer fechas da el único dato fiable: cuánto cuesta de verdad un módulo de este cliente. Conviene elegir uno de tamaño medio y con JavaScript, porque OWL es lo que más rompe.

Octubre - Noviembre 2026

Construcción

Actividad	Entregable verificable	Resp.
Localización Contable V20	Módulos instalando, log sin advertencias	MORNIX
Localización Nómina V20	Idem	MORNIX
Ajuste de funcionalidades a V20	Idem	MORNIX-RIDERY
Creación ambiente QA Odoo 20	Entorno accesible por URL propia	MORNIX-RIDERY
+ Pruebas automatizadas por módulo	Suite en verde por cada módulo migrado	MORNIX
+ Bitácora de roturas al día	Se actualiza con cada módulo	MORNIX
+ Seguimiento semanal de venders	Estado de cada módulo de tercero	MORNIX

El trabajo de estos dos meses es la repetición del ciclo de la sección siguiente, módulo por módulo, en el orden que dictó el análisis de acoplamiento.

Diciembre 2026

QA

Actividad	Entregable verificable	Resp.
Configuración de módulos en QA	Ambiente configurado como producción	MORNIX-RIDERY
Desarrollos y ajustes	Cambios con su prueba	MORNIX-RIDERY
Migración de datos de prueba	Base restaurada y actualizada sin error	MORNIX
Pruebas funcionales unitarias e integrales	Resultados con evidencia	MORNIX-RIDERY
+ Comparación contra la instancia v16	Totales de asientos, libros y reportes legales, iguales	MORNIX-RIDERY
+ Verificación de secuencias fiscales	Los correlativos continúan, no se reinician	MORNIX
+ Criterios de aceptación firmados	Anexo C, aceptado por RIDERY	RIDERY

El criterio no es «funciona», es «da lo mismo que antes»

Para eso hay que mantener la instancia v16 viva y comparar contra ella con los mismos datos. Sin ese contraste, «las pruebas pasaron» solo significa que el sistema no se cayó.

Enero - Febrero 2027 **Producción**

Actividad	Entregable verificable	Resp.
Homologación Localización V20	Aceptación formal	MORNIX
Migración de datos reales	Base de producción migrada	MORNIX-RIDERY
Pruebas finales	Resultados con evidencia	MORNIX-RIDERY
Reentrenamiento del personal	Sesiones dictadas y material entregado	MORNIX-RIDERY
Pase a producción	Sistema en operación	MORNIX-RIDERY
+ Ensayo del corte, cronometrado	Duración real de la ventana de indisponibilidad	MORNIX
+ Procedimiento de reversión escrito	Documento con pasos y punto de no retorno	MORNIX
+ Respaldo verificado antes del corte	Archivo comprobado, no solo «el comando no falló»	MORNIX
+ Acompañamiento post-arranque	Ventana de soporte reforzado definida	MORNIX

Sobre el reentrenamiento: queda en la misma ventana que el pase. Conviene adelantarlo a enero, con el ambiente de QA ya estable — capacitar sobre un sistema que todavía se está ajustando obliga a repetir la capacitación.

El ciclo por módulo

Esto es lo que se repite durante octubre y noviembre. El orden importa: cada paso descubre problemas que el siguiente ya no puede esconder.

1 · Leer antes de tocar

- Revisar el autor: si es de un tercero o de OCA, **detenerse** y escalarlo.
- Leer el módulo entero; anotar qué modelos extiende y de qué depende de verdad.
- Leer la bitácora del proyecto. Si otro módulo ya tropezó con algo, la solución está ahí. Esa tabla es lo que hace que el módulo 40 cueste menos que el módulo 3.

2 · Corregir el manifest

La versión del manifest decide si el módulo existe
v20 valida el campo `version`. Si no coincide con la serie, marca el módulo `installable=False`, deja un **WARNING** (no un error),
termina con código de salida 0
y no instala nada. Mirando el código de salida, parece éxito.

Y `20.0.1.0.0`
tampoco sirve
: la serie de `master` es `19.5`. Hay que usar la **forma corta** (`1.0.0`), a la que Odoo antepone la serie vigente. Es la única que sobrevive al release sin volver a tocar todos los manifests.

3 · Migrar la seguridad

v20 fundió `ir.model.access` e `ir.rule` en un único `ir.access`. Se convierte con herramienta (Anexo B) y se actualiza el manifest a mano, porque el orden de carga importa.

4 · Instalar, leer el error, corregir, repetir

Odoo falla de a un problema por vez y siempre en el mismo orden: imports de Python, ORM y campos, XML de seguridad, vistas, datos. Conviene no adelantarse.

Leer el log, no el código de salida
Varios modos de fallo de v20 terminan con código 0. Hay que buscar explícitamente `WARNING`, `ERROR` y `CRITICAL`.

5 · Dejar el log sin advertencias

Una advertencia nueva es una rotura que todavía no se manifestó.

6 · Correr los tests que ya existen

Si el módulo trae pruebas, son el mejor activo disponible: verifican comportamiento, no solo que instale. En el piloto sacaron a la luz cuatro roturas que la instalación no tocaba.

7 · Escribir las que faltan

Prioridad, por lo que más se rompe: campos calculados, restricciones, onchange, secuencias y correlativos, y los archivos que se presentan a la administración.

El caso que justifica esta disciplina
En el piloto, el libro fiscal **estaba roto**
tras migrar: usaba cuatro APIs que v20 eliminó. Ni la instalación ni las pruebas heredadas tocaban ese camino, así que el módulo «instalaba limpio» con el reporte inservible. Lo encontró una prueba escrita a propósito para ese flujo.

8 · Documentar

- Cada rotura nueva en la bitácora: síntoma, versión que la introdujo, solución.
- **Lo que quedó sin resolver, explícito.** Un módulo entregado con un pendiente callado cuesta mucho más caro que uno entregado con el pendiente escrito.

Migrar, no reescribir
Se conserva la lógica de negocio tal cual. Si algo parece mal diseñado, se anota pero no se cambia: el cliente valida contra el comportamiento actual, y un cambio «de mejora» metido en una migración es imposible de distinguir de una regresión.

Anexo A — Catálogo de roturas v16 → v20

Verificado: comprobado contra el código de `master` y reproducido en la migración piloto. **Esperado:** documentado por Odoo, aún no confirmado contra código de RIDERY.

A.1 Bloquean la instalación

Rotura	Estado	Solución
Versión del manifest validada; si no coincide, el módulo no se instala y el proceso termina con código 0	Verificado	Forma corta <code>x.y.z</code>
<code>ir.model.access</code> + <code>ir.rule</code> → <code>ir.access</code>	Verificado	Herramienta de conversión
<code>odoo.tools.misc</code> ya no re-exporta <code>xlswriter</code> ni <code>pytz</code>	Verificado	Importar directamente
<code>odoo/osv/</code> eliminado, incluido <code>expression</code>	Verificado	<code>odoo.orm.domains.Domain</code>
ORM reorganizado: <code>models.py</code> y <code>fields.py</code> pasan a ser paquetes	Verificado	Ajustar imports
<code>_sql_constraints</code> sin soporte	Verificado	<code>models.Constraint</code>
<code>ir.actions.report</code> perdió <code>report_file</code>	Verificado	Quitar el campo

A.2 Rompen las vistas

Rotura	Estado	Solución
<code>attrs</code> y <code>states</code> eliminados	Esperado (v17)	Atributos directos con expresión Python
<code><tree></code> renombrado a <code><list></code>	Esperado (v18)	Renombrar
<code><group></code> ya no se admite dentro de <code><search></code>	Verificado	Sacar los filtros del grupo
Un <code><label for="x"></code> huérfano invalida la vista	Verificado	Reemplazar etiqueta junto con el campo
Anclajes de <code>xpath</code> movidos en las vistas base	Verificado	Reanclar al envoltorio del campo, no a la maquetación

A.3 Cambian comportamiento sin dar error

Las más peligrosas: el módulo instala, las pruebas pasan, y el resultado es otro.

Rotura	Estado	Consecuencia
<code>name_get()</code> → <code>_compute_display_name</code>	Esperado (v17)	No da error: deja de llamarse. El modelo muestra mal su nombre en silencio
<code>res.partner.company_type</code> eliminado; <code>is_company</code> se calcula desde el identificador fiscal	Verificado	La heurística clasifica como empresa a todo contacto con RIF. En Venezuela una persona natural también lo tiene — y de eso depende la retención
<code>base_vat</code> normaliza el identificador y le quita los separadores	Verificado	Afecta reportes y archivos que lo imprimen
La validación del identificador pasó de <code>check_vat</code> al <i>inverse</i> del campo	Verificado	<code>super().check_vat()</code> lanza <code>AttributeError</code>
Los <code>Selection</code> ya no aceptan <code>''</code> como vacío	Verificado	Usar <code>False</code>
<code>account.account.deprecated</code> eliminado	Verificado	Se sustituye por <code>active</code> , invertido
<code>company_registry</code> eliminado	Verificado	Lo sustituyen los identificadores adicionales

A.4 Rompen al ejecutar, no al instalar

Aparecen cuando alguien abre el formulario o genera el reporte. Sin pruebas que recorran ese camino, llegan a producción.

Rotura	Estado	Solución
Los recordsets ya no exponen <code>_uid</code>	Verificado	<code>self.env.uid</code>
<code>check_access_rights</code> y <code>check_access_rule</code> eliminados	Verificado	<code>check_access</code>
<code>_where_calc</code> y <code>_apply_ir_rules</code> eliminados	Verificado	<code>_search</code> , que devuelve un <code>Query</code> con las reglas aplicadas
Los objetos <code>SQL</code> ya no exponen <code>.code</code> ni <code>.params</code>	Verificado	Componer objetos <code>SQL</code> y pasarlos enteros al cursor
<code>ir.config_parameter.get_param / set_param</code>	Verificado	<code>get_str</code> , <code>get_bool</code> , <code>get_int</code> , <code>get_float</code>
Las acciones de recibo del núcleo desaparecieron	Verificado	Los tipos existen, pero sin punto de entrada

A.5 Entorno

Rotura	Estado	Solución
Python mínimo 3.12 · PostgreSQL mínimo 16	Verificado	Dimensionar el servidor
<code>http_interface</code> pasa a <code>127.0.0.1</code>	Verificado	Abrirlo en el contenedor, limitar en el proxy
El envoltorio de <code>xlswriter</code> que saneaba nombres de hoja desapareció	Verificado	Nombres de hoja ≤31 caracteres y sin <code>[]:*/\</code>
El plan de cuentas hay que cargarlo explícitamente	Verificado	Instalar la localización no crea cuentas ni diarios

Anexo B — Herramientas

Herramienta	Qué resuelve
Conversor de seguridad	Convierte <code>ir.model.access.csv</code> y las reglas <code>ir.rule</code> al <code>ir.access.csv</code> de v20. Resuelve los nombres de modelo contra los declarados en el código, no deduciéndolos del identificador externo — que falla con modelos que llevan guión bajo.
Analizador de dependencias	Encuentra acoplamiento que los manifests no declaran. Da el orden real de migración.
Analizador de cobertura	Lista los modelos que ninguna prueba menciona. Sirve para encontrar huecos, no para declarar cobertura.
Generador de pruebas	Genera esqueletos por modelo. Los casos fallan a propósito : un esqueleto en verde se confunde con cobertura real.
Generador de recorridos de interfaz	Única forma práctica de verificar las vistas migradas a OWL.

Anexo C — Criterios de aceptación

Un módulo pasa a desplegable solo si cumple **todos**. Un criterio que no se pudo verificar es un criterio **no cumplido**, no un aprobado con nota al pie.

C.1 Técnicos

#	Criterio
A1	Instala en base limpia sin error
A2	Actualiza sobre base con datos sin error
A3	Las pruebas del módulo pasan
A4	El log no tiene <code>ERROR</code> ni <code>CRITICAL</code> nuevos
A5	El log no tiene advertencias nuevas sin justificar
A6	Las vistas cargan sin error de validación
A7	Los reportes PDF se generan
A8	No quedan <code>attrs</code> , <code>states</code> ni <code><tree></code> en el XML

C.2 De datos

#	Criterio
B1	Los datos existentes siguen accesibles tras actualizar
B2	Los campos que cambiaron de tipo o nombre tienen script de migración
B3	Los asientos contables cuadran igual que antes
B4	Las secuencias fiscales no se reinician ni saltan
B5	Los montos en moneda extranjera dan igual

C.3 De negocio — los confirma RIDERY

#	Criterio
C1	Los flujos de uso diario funcionan igual que antes
C2	Los reportes legales dan idéntico
C3	La impresión fiscal funciona con el hardware real
C4	El punto de venta opera completo, incluyendo pagos y vuelto
C5	Los usuarios conservan sus permisos

Lo que un veredicto favorable no cubre

No garantiza que se haya probado con el volumen real de datos, ni que los módulos de terceros existan ya en v20, ni que la interfaz se vea igual: OWL cambió el frontend y algunas diferencias visuales son inevitables. Esas tres cosas se validan aparte, con RIDERY, antes de comprometer la fecha de salida.